19. API Testing Framework Structure

The **API Testing Framework Structure** defines how the entire automation setup is organized, designed, and executed for the project using Fake Store API.

This structure ensures the framework is:

- Scalable
- Maintainable
- Reusable
- Automation-ready (Postman + Newman CLI)
- CI/CD compatible

19.1 What is API Testing Framework?

An API testing framework is a **structured setup of tools, folders, scripts, and configurations** used to:

- Create API test cases
- Execute them automatically
- Validate responses
- Generate reports

In this project, we built a **Postman + Newman based hybrid framework**.

19.2 High-Level Framework Architecture

◆ Layers of the framework:

1. Test Design Layer

- Postman Collections
- API Requests (GET, POST, PUT, DELETE)
- Test Scripts (JavaScript assertions)

---

2. Configuration Layer

- Environment JSON file
- Global variables
- Base URL setup

- Authentication token handling

---

3. Execution Layer

- Postman Collection Runner (manual execution)
- Newman CLI (automation execution)

---

4. Validation Layer

- Status code validation
- Response body validation
- JSON schema checks
- Response time validation

---

5. Reporting Layer

- Newman CLI reports
- HTML Extra reports
- Execution logs
- Test summary reports

19.3 Project Folder Structure

```
API Testing Framework
│
├── 1. Postman Collection
│    ├── Authentication
│    ├── Products
│    ├── Cart
│    ├── Users
│    └── Categories
│
├── 2. Environment Files
│    ├── QA Environment.json
│    └── Dev Environment.json
│
├── 3. Test Scripts
│    ├── Status Code Validation
│    ├── Response Body Validation
│    ├── Schema Validation
│    └── Token Handling Scripts
│
├── 4. Execution
```

```
│    ├── Collection Runner
│    └── Newman CLI Commands
│
├── 5. Reports
│    ├── Newman CLI Output
│    ├── HTML Reports
│    └── Execution Summary
│
└── 6. Documentation
     ├── Test Cases
     ├── Test Scenarios
     ├── Bug Reports
     └── Test Closure Report
```

19.4 Key Components Explained

◆ 1. Postman Collection

- Core of the framework
- Contains all API requests
- Organized in modules (Products, Cart, Users)

◆ 2. Environment Management

- Stores dynamic values:
  - baseURL
  - authToken
  - productId
- Enables multi-environment testing

◆ 3. Test Scripts (Validation Engine)

Used inside Postman:

- Status code checks
- JSON validation
- Response time checks
- Data verification

◆ 4. Newman CLI Integration

- Converts Postman into automation tool
- Executes collections via terminal
- Supports CI/CD pipelines

◆ 5. Reporting System

- HTML reports for visualization
- CLI logs for debugging
- Execution summary for QA review

19.5 Execution Flow of Framework

Step-by-step flow:

1. API request triggered (Postman/Newman)
2. Request sent to Fake Store API
3. Response received
4. Validation scripts executed
5. Results recorded (Pass/Fail)
6. Reports generated (CLI + HTML)

19.6 Framework Advantages

- Easy to maintain
- Reusable across projects
- Supports automation (Newman)
- CI/CD compatible
- Scalable for large API suites
- Reduces manual effort

19.7 Real-World Use Case

In real companies:

- Developers deploy APIs
- QA runs Newman in CI/CD pipeline
- Reports are generated automatically
- Failures trigger build failure
- Teams fix issues quickly
